Ref: 2026/II3 **Presented during June 2026 session**

University of Manouba
National School of Computer Science
(ENSI)

Final Graduation Project Report

Presented for the purpose of obtaining the title of

ENGINEER IN COMPUTER SCIENCE

By: Oussema Aissaoui

Agentic AI Recruiter: A Multi-Agent Platform for Multimodal Interview Analysis and Recruiter-Adaptive Decision Support

---

Host Organization:

Head of department: Mrs. Imen Ayari

Academic Supervisor: Mrs. Dorra Dhouib

Company Supervisors: Mrs. Yosr Omrane, Mrs. Hayfa Grayet

Address: 10 Rue de l'énergie solaire Impasse N°1 Charguia 1, Tunis 2035

Academic Year: 2025–2026

Appreciation and Supervisor's Signature

Dedication

This work is the reflection of the boundless love and unwavering support that have illuminated every step of my life and of my academic journey.

To my beloved father, **Mondher**, you have been the quiet strength behind every milestone I have reached. From the earliest days of my schooling to the choice of the most demanding schools and universities, you stood by me with patience, sacrifice and a steady hand. The values of discipline, perseverance and integrity that guide me today are nothing more than the daily example you have set. No words could ever match the gratitude I owe you.

To my dearest mother, **Hajer**, you are the warmth and the soul of our home. Your tender care, your endless patience and the countless nights you spent watching over my studies have shaped the person I have become. Even now, you continue to surround me with attention so that nothing is ever missing from my path. Your love is the most precious gift of my life, and for that I will be forever thankful.

To my elder sister, **Minyar**, your kindness, complicity and constant encouragement have been a true blessing. As you prepare for the *baccalauréat*, I wish you the courage, the calm and the success you deserve, and may this milestone open the doors of the future you dream of.

To my little sister, **Maram**, your laughter and your spontaneous affection bring light to every ordinary day. As you face your ninth-grade national exam, I wish you confidence, focus and a brilliant result, and I have no doubt that you are about to make us all proud.

To my dear friends, my faithful companions through the highs and lows of these years, your loyalty, your humour and the moments we have shared have made this journey infinitely richer. Your support has been quietly woven into every success I have reached, and I am truly grateful for each of you.

To all of you, with love and gratitude, I dedicate this humble work.

Oussema Aissaoui

Acknowledgements

The completion of this graduation project owes much to the people who guided, supported and trusted me throughout the internship, and I am glad to take this opportunity to express my sincere gratitude.

I would first like to thank my academic supervisor, **Mrs. Dorra Dhouib**, from the *École Nationale des Sciences de l'Informatique (ENSI)*, for her attentive guidance, her constructive feedback and the academic rigour with which she shaped the direction of this work.

I would also like to express my sincere appreciation to **Mrs. Imen Ayari**, head of the Innovation Factory department, for the welcoming environment, the institutional support and the trust placed in me throughout this internship.

My deepest thanks go to my company supervisors at *Talan Tunisie Consulting*, **Mrs. Yosr Omrane** and **Mrs. Hayfa Grayet**, whose technical expertise, availability and trust were decisive at every milestone of the project. Their thoughtful insights consistently elevated the quality of the proposed solution.

My gratitude further extends to the entire team of the *Innovation Factory* at Talan Tunisie Consulting, whose collaborative spirit, technical generosity and daily encouragement created an inspiring setting in which to learn, build and grow.

Finally, I wish to thank the members of the jury for the honour of evaluating this work and for the time devoted to its review.

Contents

Introduction **1**

1 General Context **2**

1.1 Host Organisation 2

1.1.1 Talan Group 2

1.1.2 Talan Tunisie Consulting 2

1.1.3 Business Sectors 2

1.1.4 Strategic Partnerships 3

1.1.5 Talan Innovation Factory 3

1.1.6 The Talan Bootcamp 3

1.2 Project Overview 3

1.2.1 Project Context 3

1.2.2 Problem Statement 4

1.2.3 Objectives 4

1.3 Review of Existing Solutions 5

1.3.1 Job Description Authoring 5

1.3.2 Sourcing and Candidate Matching 5

1.3.3 Conversational and Asynchronous Interviewing 5

1.3.4 Comparative Synthesis 6

1.4 Proposed Solution 6

1.5 Project Management Methodology 9

1.5.1 Why Scrum? 9

1.6 Data-Science Methodology 10

1.6.1 Why CRISP-DM? 11

1.7 Conclusion 11

2 State of the Art **12**

2.1 Introduction 12

2.2 Foundations of Artificial Intelligence and Natural Language Processing . . 12

2.2.1 Artificial Intelligence, Machine Learning and Deep Learning 12
2.2.2 Learning Regimes 13
2.2.3 Pre-training, Fine-Tuning and Transfer Learning 13
2.2.4 Tokenisation and Embeddings 13
2.2.5 The Attention Mechanism 13
2.3 Large Language Models and Fine-Tuning 14
2.3.1 The Transformer Architecture 14
2.3.2 Large Language Models 15
2.3.3 Parameter-Efficient Fine-Tuning 15
2.4 Multi-Agent Systems and Agent Protocols 16
2.4.1 Agent Definition 16
2.4.2 The ReAct Framework 16
2.4.3 Multi-Agent Orchestration 17
2.4.4 Agent Communication Contracts 17
2.5 Knowledge Graphs and Graph-Augmented Retrieval 18
2.5.1 Knowledge Graphs 18
2.5.2 Retrieval-Augmented Generation 18
2.5.3 Graph RAG 18
2.6 Candidate–Job Matching 18
2.6.1 Sentence Embeddings and Semantic Similarity 19
2.6.2 Topic-Aware Person–Job Fit (TAPJFNN) 19
2.6.3 Latent Dirichlet Allocation 19
2.6.4 Graph Neural Networks 19
2.6.5 Genetic Algorithms for Constrained Shortlisting 20
2.7 Affective Computing and Speech Technologies 21
2.7.1 Models of Emotion 21
2.7.2 Facial Action Units and Landmarks 21
2.7.3 Self-Supervised Speech Encoders 21
2.7.4 ASR, TTS and Voice-Activity Detection 22
2.8 Personalised Ranking 22
2.9 Conclusion 22

3 Analysis and Design 23
3.1 Requirements Analysis 23
3.1.1 Identification of the Different Actors 23
3.1.2 Functional Requirements 24
3.1.3 General Use Case Diagram 24
3.1.4 Non-Functional Requirements 25
3.2 Project Planning with Scrum 26

3.2.1 Definition of Roles 26
3.2.2 Product Backlog 27
3.2.3 Sprint Planning 28
3.3 Application Architecture 28
3.3.1 Physical Architecture 28
3.3.2 System Logical Architecture 29
3.3.3 Logical Architecture of the Multi-Agent System 30
3.4 Work Environment 32
3.4.1 Hardware Environment 32
3.4.2 Software Environment 33

Conclusion et perspectives **34**

A Annexe 1 **35**

Bibliography **35**

Netography **36**

List of Figures

1.1 Pipeline of the proposed platform. 7
1.2 High-level system architecture of HireFlow. 8
1.3 Overview of the Scrum framework. 9
1.4 Overview of the CRISP-DM lifecycle. 10

2.1 Transformer encoder–decoder architecture. 15
2.2 ReAct thought–action–observation loop. 17
2.3 TAPJFNN four-level topic-aware hierarchical attention. 19
2.4 GNN message passing on a bipartite candidate–job graph. 20

3.1 General use-case diagram of HireFlow. 25
3.2 Physical deployment architecture of HireFlow. 29
3.3 Three-tier logical architecture of HireFlow. 30
3.4 Logical architecture of the HireFlow multi-agent system. 31

List of Tables

1.1 Capability coverage across reviewed platforms. 6

3.1 Functional requirements traced by epic. 24
3.2 Scrum roles and their assignment. 26
3.3 Product backlog (M = Must, S = Should, C = Could). 27
3.4 Six-sprint roadmap of the HireFlow internship. 28
3.5 Hardware environment used during the internship. 32
3.6 Software environment used during the internship. 33

List of Algorithms

1 Genetic algorithm for constrained shortlisting 21

List of acronyms

A2A *Agent-to-Agent* (protocol)

AI *Artificial Intelligence*

API *Application Programming Interface*

ASR *Automatic Speech Recognition*

ATS *Applicant Tracking System*

AU *Action Unit*

BERT *Bidirectional Encoder Representations from Transformers*

BPE *Byte-Pair Encoding*

CPU *Central Processing Unit*

CRISP-DM *Cross-Industry Standard Process for Data Mining*

CRUD *Create, Read, Update, Delete*

CV *Curriculum Vitae*

DL *Deep Learning*

ENSI *École Nationale des Sciences de l'Informatique*

EU *European Union*

FACS *Facial Action Coding System*

GA *Genetic Algorithm*

GAT *Graph Attention Network*

GCN *Graph Convolutional Network*

GIN *Graph Isomorphism Network*

GNN *Graph Neural Network*

GPU *Graphics Processing Unit*

HF *HuggingFace*

HITL *Human-In-The-Loop*

HMM *Hidden Markov Model*

HR *Human Resources*

JD *Job Description*

JSON *JavaScript Object Notation*

KG *Knowledge Graph*

LDA *Latent Dirichlet Allocation*

LLM *Large Language Model*

LoRA *Low-Rank Adaptation*

LR *Logistic Regression*

LTS *Long-Term Support*

MCP *Model Context Protocol*

ML *Machine Learning*

MoSCoW *Must, Should, Could and Won't have* (prioritisation)

NIST *National Institute of Standards and Technology*

NLP *Natural Language Processing*

ORM *Object-Relational Mapping*

PDF *Portable Document Format*

PEFT *Parameter-Efficient Fine-Tuning*

PFE *Projet de Fin d'Études*

QLoRA *Quantised Low-Rank Adaptation*

R3F *React Three Fiber*

RAG *Retrieval-Augmented Generation*

RAM *Random-Access Memory*

RBAC *Role-Based Access Control*

RDF *Resource Description Framework*

RMF *Risk Management Framework*

SPA *Single-Page Application*

SQL *Structured Query Language*

SSD *Solid-State Drive*

TAPJFNN *Topic-Aware Person–Job Fit Neural Network*

TTS *Text-to-Speech*

VAD *Voice-Activity Detection*

VRAM *Video Random-Access Memory*

Introduction

In recent years, the integration of artificial intelligence and machine learning has profoundly reshaped the way organisations manage their human-capital decisions. With the rise of large language models, multimodal vision and speech encoders and agentic frameworks, machines now understand documents, conduct conversations and analyse behaviour with unprecedented fluency. Yet, despite this progress, the application of these techniques to recruitment, one of the most consequential and most regulated decisions an organisation makes, remains fragmented, shallow and largely opaque.

Hiring teams currently rely on a patchwork of disconnected products: applicant-tracking systems perform keyword filtering, sourcing platforms expose closed proprietary scores, and asynchronous video tools record candidates without ever adapting to them. At the same time, the European AI Act and the NIST AI Risk Management Framework now classify employment-related AI as *high-risk* and require a documented decision trail and meaningful human oversight at every stage.

The goal of this project, titled **HireFlow**, is to design and deliver an end-to-end agentic platform that supports a recruiter throughout the full hiring journey, from a vacancy opening to the final hiring decision. Built around five collaborating agents responsible for culture-aware job-description authoring, candidate–job matching, conversational interviewing, multimodal behavioural analysis and recruiter-adaptive scoring, the system leverages transformer models, graph neural networks and computer-vision pipelines while preserving meaningful human oversight at every step.

This report is structured as follows. The first chapter situates the project, reviews existing platforms and presents the proposed multi-agent solution. The remaining chapters follow the Scrum cadence of the internship — one chapter per sprint — and the sprints that build the artificial-intelligence agents are themselves organised around the six phases of CRISP-DM. The final sections discuss the results, limitations and perspectives for future improvement.

Chapter 1

General Context

This chapter situates the project within its operational context. It introduces the host organisation, frames the recruitment landscape, states the problem and objectives, reviews existing platforms, presents the proposed multi-agent solution and describes the project-management methodology adopted during the internship.

1.1 Host Organisation

1.1.1 Talan Group

Talan is an international consulting and technology company founded in 2002 by Mehdi Houas, Eric Benamou and Philippe Cassoulat. The group leverages information and communication technologies to reshape how organisations interact with their customers, data and operations. Its activities span four strategic verticals: financial services and insurance, telecommunications and media, energy and utilities, and transport and logistics [URL7]. Over two decades the group has grown from a Paris boutique into a worldwide network of more than one thousand consultants, with offices in France, the United Kingdom, the United States, Hong Kong and Canada.

1.1.2 Talan Tunisie Consulting

Talan Tunisie Consulting is the group's nearshore competence centre. Its roughly one hundred and eighty engineers collaborate with European customers on demanding programmes ranging from cloud-native software engineering to artificial-intelligence research. The internship reported here was conducted in this entity.

1.1.3 Business Sectors

The group's activities are organised around four core verticals.

- **Financial services and insurance**: digital channels, customer-journey redesign and data-driven risk controls.

- **Telecommunications and media**: personalised services, modernised network architectures and resilient distribution platforms.
- **Energy and utilities**: smart metering and low-carbon infrastructures.
- **Transport and logistics**: real-time tracking, predictive maintenance and connected supply chains.

1.1.4 Strategic Partnerships

Talan delivers through a network of partnerships with the dominant providers of enterprise technology: Microsoft, Oracle, Salesforce, Amazon Web Services, Atlassian and IBM. These alliances open access to preferential roadmaps, training and certification tracks that are reinjected into client engagements.

1.1.5 Talan Innovation Factory

The Talan Innovation Factory was launched in 2017 inside the Tunisian entity under the supervision of Mrs. Imen Ayari. It acts as an internal research and development cell that turns emerging research into business-grade prototypes. Its work programme covers artificial intelligence, quantum computing, blockchain, the Internet of Things and data science.

1.1.6 The Talan Bootcamp

New interns at the Innovation Factory follow a structured five-week bootcamp. Each week targets a specific topic, from rapid prototyping to advanced machine learning, and concludes with the delivery of a working artefact defended in front of a jury. This mechanism ensures a common technical baseline before long-term assignments.

1.2 Project Overview

1.2.1 Project Context

Recruitment teams operate under intense pressure: open positions attract more applications than human reviewers can read, while candidates expect faster and more transparent decisions. Regulators now require that hiring processes be auditable and that automated decisions be explainable. The European Artificial Intelligence Act and the NIST AI Risk Management Framework classify employment-related artificial intelligence as *high-risk*, mandating a documented decision trail and meaningful human oversight at every stage. In this environment, keyword-based applicant tracking is no longer sufficient, and the profession is gradually reorganising itself around assistive AI tooling.

The present project, conducted within the Innovation Factory, designs an end-to-end agentic platform that supports a recruiter throughout the hiring journey, from a vacancy opening to the final hiring decision. Rather than a monolithic application, the work explores how independent specialised agents can collaborate to author the job description, screen incoming curricula, conduct the interview, analyse behaviour and produce an explainable scoring report.

1.2.2 Problem Statement

Current hiring workflows exhibit several recurring shortcomings.

- **Generic, culture-blind job descriptions**: most JDs are written from outdated templates and ignore what really characterises a successful hire in the company, often perpetuating biased phrasings.
- **Shallow candidate–job matching**: keyword overlap fails to capture transferable skills, semantic equivalence between titles or context-dependent experience.
- **Limited interaction in asynchronous interviews**: one-way video tools record candidates without follow-up questions or adaptation to the candidate's profile.
- **Loss of behavioural signal in remote settings**: engagement, hesitation and confidence cues that recruiters naturally read in person are discarded by current remote tooling.
- **Fragmented tooling**: authoring, screening, interviewing and scoring live in disconnected products, which prevents learning loops between stages.
- **Opacity and lack of personalisation**: scores are presented as a single monolithic number and ignore the fact that different recruiters legitimately weight signals differently.

These observations call for a unified system that combines culture-aware JD authoring, deep semantic matching, adaptive conversational interviewing, multimodal behavioural analysis and explainable, recruiter-aware scoring.

1.2.3 Objectives

- **Culture-aware JD authoring**: an autonomous agent that grounds its generation in a knowledge graph of the company's talent landscape, retrieves the relevant subgraph through Graph RAG, and produces a personalised JD protected by a bias and compliance review.
- **Semantic matching**: a model that fuses textual encoders, hierarchical attention and graph-based representations of the candidate–position bipartite structure.
- **Conversational interviewing**: a virtual recruiter that grounds its dialogue in the candidate's CV and decides between follow-up and topic-switching strategies.

- **Multimodal behavioural analysis**: a low-dimensional profile extracted from video and audio streams, covering cognitive load, emotional arousal, engagement and confidence.
- **Objective scoring and reporting**: structured post-interview reports combining transcript analysis, behavioural metrics and explicit hiring recommendations.
- **Recruiter-adaptive decision support**: a personalised signal that learns each recruiter's decision patterns while keeping the human in control.
- **Unified delivery platform**: integrate the previous components into a single web application.

1.3 Review of Existing Solutions

This section positions the most representative platforms and highlights the gap that the proposed system addresses.

1.3.1 Job Description Authoring

Textio and **Ongig** flag gendered terms and exclusionary expressions in an existing draft and propose neutral alternatives. Their contribution stays at the stylistic-polishing layer: neither generates the structural content of a description, neither taps into the company's workforce data, and neither manages the human-review checkpoints required by high-risk AI regulation.

1.3.2 Sourcing and Candidate Matching

LinkedIn Recruiter exposes a powerful search engine over the LinkedIn graph, but its matching logic is not customisable by the hiring team [URL4]. **Eightfold AI** encodes profiles in a vector space supporting skill-based search and internal mobility, yet ships as a closed enterprise product whose scoring internals are not exposed [URL2]. **Manatal** and **Workable** bundle a candidate database with lightweight keyword-based matching and compete on ergonomics rather than algorithmic depth [URL5].

1.3.3 Conversational and Asynchronous Interviewing

HireVue is the market reference for asynchronous video interviewing: candidates record themselves answering predefined questions and the platform extracts features for automated rating [URL3]. The format scales but precludes any adaptation of the questions to the candidate. **Modern Hire** (now part of HireVue) and **myInterview** share the same non-conversational paradigm and the same criticisms regarding scoring transparency.

1.3.4 Comparative Synthesis

Table 1.1 summarises how the reviewed platforms cover the five target capabilities. Each platform excels in a narrow band, but none simultaneously offers culture-aware JD authoring, deep semantic matching, adaptive conversation, behavioural analysis and recruiter-level personalisation.

Platform	**Culture-aware JD**	**Deep matching**	**Conversational**	**Behavioural**	**Recruiter-adaptive**
Textio / Ongig	Partial	No	No	No	No
LinkedIn Recruiter	No	Partial	No	No	No
Eightfold AI	No	Yes	No	No	No
Manatal / Workable	No	No	No	No	No
HireVue	No	No	No	Partial	No
Modern Hire	No	No	No	Partial	No
Proposed system	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

Table 1.1: Capability coverage across reviewed platforms.

1.4 Proposed Solution

The proposed solution, internally named *Agentic AI Recruiter*, is a multi-agent platform that orchestrates the full hiring journey under a single web application. Specialised agents communicate through a lightweight agent-to-agent protocol exposed by a unified backend, and the platform is structured around five operational stages summarised in Figure 1.1.

(1) Job Description Authoring. A dedicated JD agent builds a *knowledge graph* of the company's talent landscape (employees, skills, projects, culture markers) and uses *Graph RAG* to retrieve the relevant subgraph — past successful hires, shared skill clusters and prevailing culture cues — which is then fed to a large language model that drafts a personalised, culture-aware JD. The output is reviewed by a human through an evidence panel before publication, in line with the EU AI Act's human-oversight requirements. The full multi-agent crew and its knowledge-graph backend are planned for the next iteration; the current implementation already exposes a basic JD-generation endpoint.

(2) Candidate–Job Matching. CVs and the generated JD are encoded with sentence transformers, fed to a hierarchical-attention neural ranker and refined by a graph neural network on the bipartite candidate–skill structure. The output is a ranked shortlist that the recruiter validates.

(3) Conversational Interview. A fine-tuned large language model drives the dialogue under a recruiter persona, generates contextual questions grounded in the candidate's CV and adapts between follow-up and topic-switching according to the level of detail in each answer. A three-dimensional avatar with lip-synced speech softens the asynchronous feel.

(4) Behavioural Analysis. Webcam frames feed facial-landmark and pose-estimation models, complemented by an emotion classifier and an audio encoder. The features are fused into a four-dimensional profile (cognitive load, emotional arousal, engagement and confidence) calibrated against an online baseline of the candidate's first seconds of speech.

(5) Scoring and Reporting. A scoring agent combines transcript, behavioural profile and matching evidence into per-dimension ratings and an overall recommendation. A complementary recruiter-adaptive module learns, from each recruiter's historical decisions, a personalised logistic regression that surfaces an explainable secondary signal alongside the central recommendation without ever overriding it.

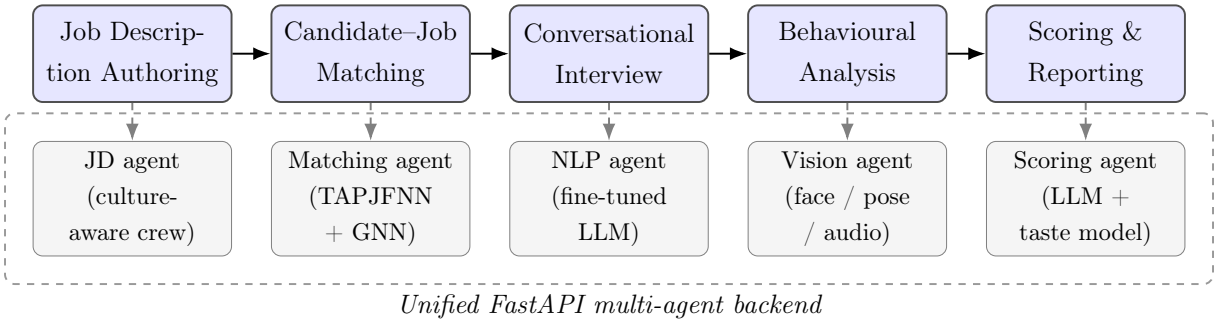

Figure 1.1: Pipeline of the proposed platform.

The platform is exposed through a unified web interface called *HireFlow*, served by a single FastAPI backend so that each agent can be developed, tested and replaced independently while the user-facing experience remains consistent. The technical decomposition is summarised in Figure 1.2, which arranges the components in five horizontal tiers. The presentation tier hosts the recruiter's browser and the HireFlow web application, built with React, Vite and Three.js for the interactive avatar. The application tier exposes the FastAPI backend, which routes traffic across the agents through an in-process agent-to-agent (A2A) dispatcher and a LangGraph orchestrator, with WebSocket streams for live vision and voice. The agent tier hosts the specialised agents. The model tier groups the AI components actually used: a self-hosted vLLM serving the fine-tuned Llama-3.1-8B for the conversational interview, the Groq API serving Llama-3.3-70B for the final scoring report, sentence-transformer encoders for matching, the MediaPipe and HSEmotion vision stack and the Kokoro and Moonshine speech pipeline. The per-recruiter logistic-regression taste model that personalises the recommendation lives inside the Recruit agent and operates on the same SQLite database. The data tier is intentionally minimal: SQLite for the relational records, the local filesystem for video, audio and PDF artefacts, and structlog for JSON-structured execution logs.

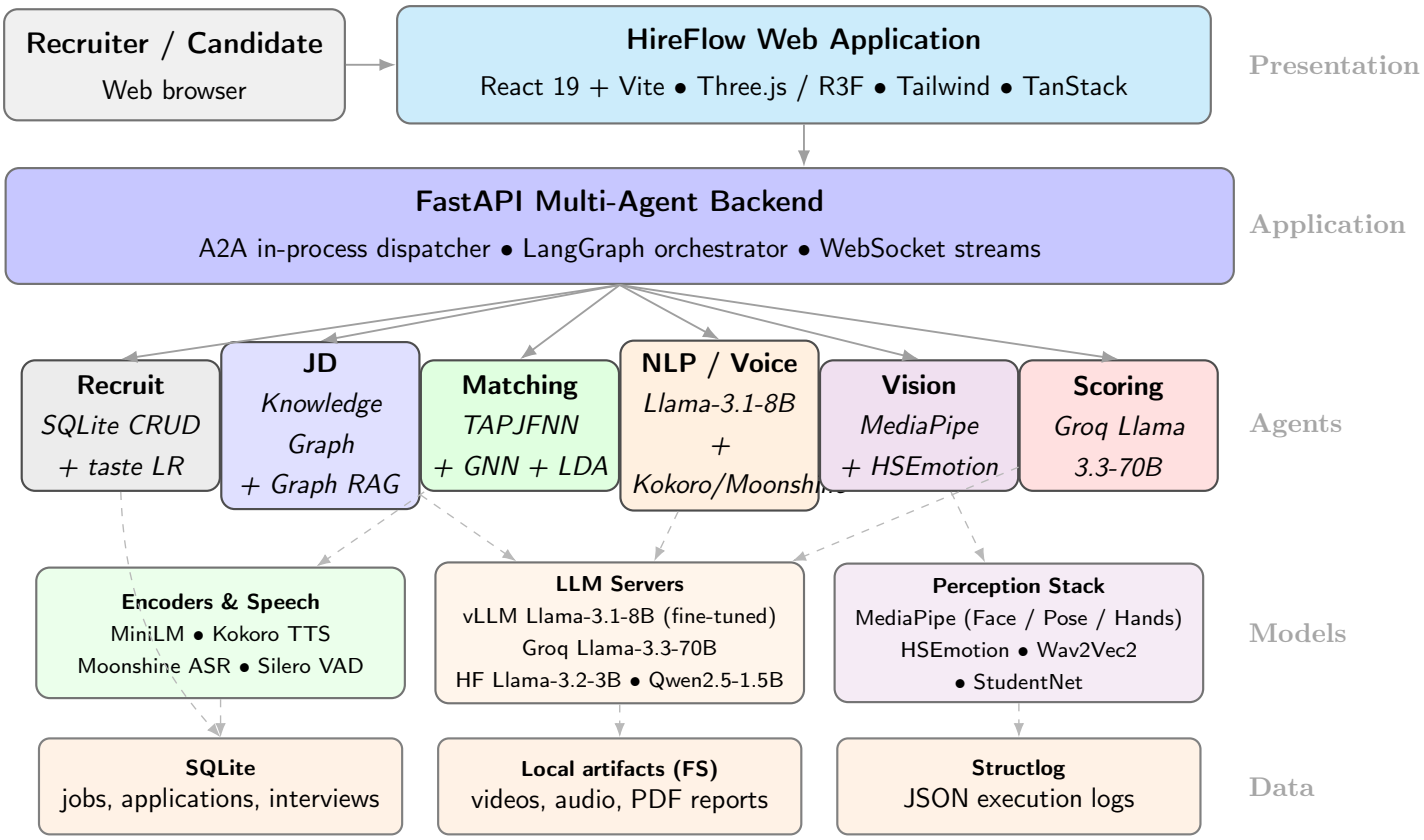

Figure 1.2: High-level system architecture of HireFlow.

1.5 Project Management Methodology

The internship was conducted under the **Scrum** framework, an iterative agile methodology that organises work into short, time-boxed sprints with a fixed cadence of planning, daily synchronisation, review and retrospective ceremonies. The three canonical roles are the Product Owner, the Scrum Master and the Development Team. Figure 1.3 provides a high-level overview of the framework, adapted from [URL6].

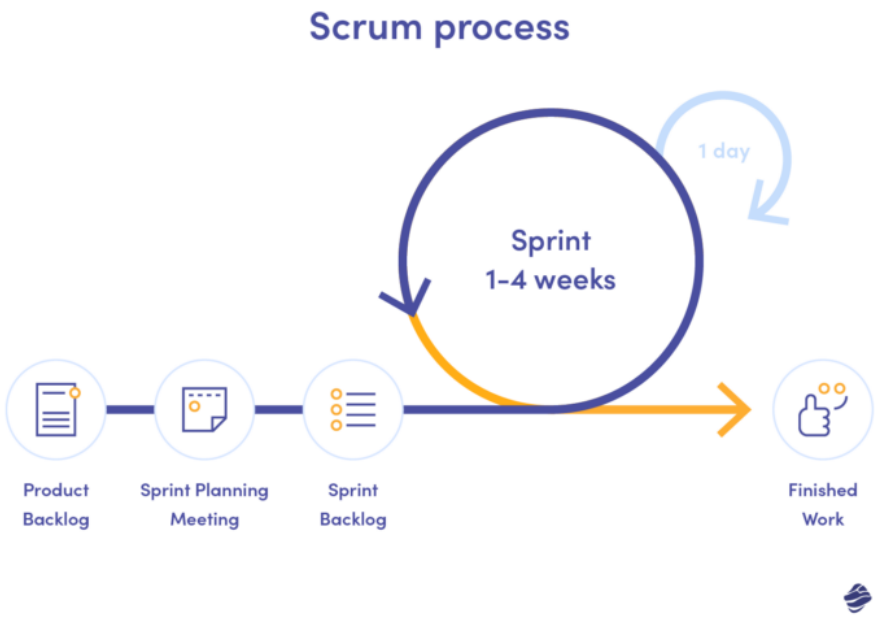

Figure 1.3: Overview of the Scrum framework.

1.5.1 Why Scrum?

Scrum was preferred over more rigid methodologies, such as Waterfall or the V-Model, for several reasons that match the nature of this project.

- **High research uncertainty**: building five artificial-intelligence agents involves frequent experimentation with models, datasets and architectures whose outcomes cannot be planned in detail months ahead. Sprints absorb this uncertainty through iteration-level re-planning.
- **Continuous feedback loop**: sprint reviews expose intermediate prototypes to the academic and industrial supervisors, which lets early misalignments be corrected before they propagate.
- **Alignment with the bootcamp cadence**: the Talan bootcamp itself follows a one-week-per-topic format with end-of-week defences, which integrates naturally with the Scrum sprint rhythm.

- **Visibility on progress**: a maintained backlog and a burndown chart give the supervisors a quantitative view of the work completed and of the remaining scope, which is critical under a fixed academic deadline.
- **Inspect-and-adapt culture**: retrospectives encourage explicit reflection on what worked at each sprint and support the continuous improvement of personal engineering practices throughout the internship.

The project relied on two-week sprints, a backlog hosted in a project-management tool, weekly synchronisations with the supervisors and end-of-sprint reviews that produced demonstrable artefacts.

1.6 Data-Science Methodology

The artificial-intelligence components of the platform — the TAPJFNN matching ranker, the candidate–skill graph neural network and the supervised fine-tuning of the conversational large language model — were developed under the **CRISP-DM** (Cross-Industry Standard Process for Data Mining) methodology. CRISP-DM organises a data-science project around six iterative phases that together cover the lifecycle from the business question to the deployment of the trained model. Figure 1.4 illustrates its cyclic structure [URL1].

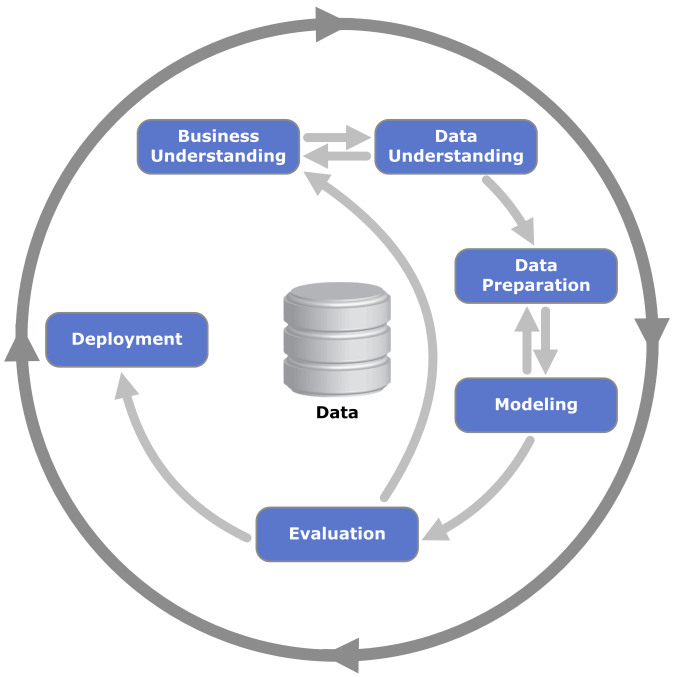

Figure 1.4: Overview of the CRISP-DM lifecycle.

1.6.1 Why CRISP-DM?

CRISP-DM was preferred over a purely engineering-driven workflow because the AI components of HireFlow are data-bound and require explicit handling of dataset assumptions, leakage controls, distribution shift and offline evaluation. The same six phases were applied independently to each of the three models, while still leaving room for the back-and-forth between modelling and data preparation that is typical of deep-learning iteration cycles.

- **Business understanding**: each model was framed against a concrete recruiter-facing metric — top-k matching accuracy for TAPJFNN, skill-graph link prediction for the GNN, instruction-following quality and JSON schema adherence for the fine-tuned LLM — before any data was collected.
- **Data understanding**: exploratory analyses surfaced class imbalance, missing-skill artefacts and recruiter-bias signals that would otherwise have leaked into the trained models.
- **Data preparation**: deduplication, anonymisation, train/validation/test stratification by job family and balanced negative-sampling strategies were defined per model.
- **Modelling**: each model was iterated under shared experiment-tracking conventions, with hyper-parameter sweeps logged centrally for reproducibility.
- **Evaluation**: held-out metrics were complemented by an offline replay against historical hiring decisions and by a fairness audit on protected attributes.
- **Deployment**: each trained artefact was wrapped behind the same FastAPI agent contract, versioned and shadow-tested before replacing the previous model in production.

The combination of Scrum at the project level and CRISP-DM at the data-science level allowed managerial agility and methodological rigour to coexist throughout the internship.

1.7 Conclusion

This chapter introduced the host organisation, motivated the project, surveyed existing tooling, presented the proposed multi-agent platform and described the project-management (Scrum) and data-science (CRISP-DM) methodologies under which the work was conducted. The remaining chapters detail the technical foundations, design and experimental validation of each agent.

Chapter 2
State of the Art

2.1 Introduction

The HireFlow project relies on a heterogeneous stack of artificial-intelligence and natural-language-processing techniques to build an end-to-end agentic recruitment assistant. This chapter introduces the key concepts, architectures and protocols on which the platform is built: the foundational notions of machine learning, transformer-based language models and parameter-efficient fine-tuning, multi-agent systems, knowledge graphs and graph-augmented retrieval, candidate–job matching, affective computing and speech technologies, and personalised ranking.

2.2 Foundations of Artificial Intelligence and Natural Language Processing

This section recalls the building blocks shared by every other component of HireFlow: the AI/ML/DL hierarchy, the three learning regimes, transfer learning, embeddings, and the attention mechanism that underlies the transformer architecture.

2.2.1 Artificial Intelligence, Machine Learning and Deep Learning

Artificial intelligence (AI) is the broad scientific field concerned with the design of computational systems that perform tasks usually associated with human intelligence: perception, reasoning, decision-making, language understanding and creative production. *Machine learning* (ML) gathers the methods inside this field that improve their behaviour by inferring patterns from data rather than by following hand-written rules. *Deep learning* (DL) is the subset of machine learning that relies on artificial neural networks with many

layers, trained end-to-end by gradient descent, and is responsible for most of the progress in vision, speech and language over the past decade.

2.2.2 Learning Regimes

Three learning regimes coexist in modern systems:

- *Supervised learning* trains a model on input–output pairs (x, y) produced by human annotation, so that it learns to predict the correct y for an unseen x.
- *Self-supervised learning* replaces the human label with a pretext task derived from the data itself, such as predicting the next token of a sentence or the masked word inside a paragraph; it is the dominant paradigm for pre-training large language and speech models on corpora too vast to annotate by hand.
- *Unsupervised learning* discovers structure in unlabelled data through clustering, dimensionality reduction or density estimation.

2.2.3 Pre-training, Fine-Tuning and Transfer Learning

Transfer learning reuses a model trained on one task to solve a related one. In modern natural-language processing, the canonical form is *pre-training* a large model on a generic self-supervised task with a very large corpus, then *fine-tuning* it on a small labelled dataset that targets the application. Pre-training amortises the cost of learning general linguistic competence, while fine-tuning specialises the model on the downstream task at a fraction of that cost.

2.2.4 Tokenisation and Embeddings

A *tokeniser* converts a string into a sequence of integer identifiers from a fixed vocabulary, usually through subword algorithms such as BPE, WordPiece or SentencePiece, which eliminate out-of-vocabulary words. An *embedding* is a fixed-size dense vector that represents a discrete object — a token, a sentence, a document or an entity — in a continuous space where geometric proximity reflects semantic similarity. *Sentence embeddings* aggregate the contextual embeddings of a whole sentence into a single vector, typically by mean-pooling, and the cosine similarity between two such embeddings is the standard semantic-distance measure.

2.2.5 The Attention Mechanism

The *attention* mechanism lets every position of a sequence selectively gather information from any other position, weighted by a learned compatibility score. In its scaled-dot-product form, three projections of the input — the queries Q, the keys K and the values

V — are combined as

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V, \tag{2.1}$$

which produces a context-sensitive update for each position. *Self-attention* restricts the operation to a single sequence; *cross-attention* lets one sequence attend to another; and *multi-head* attention runs several such operations in parallel and concatenates their outputs.

2.3 Large Language Models and Fine-Tuning

This section presents the transformer architecture, the family of large language models built on top of it, and the parameter-efficient methods that adapt them to specialised tasks without re-training the billions of base weights.

2.3.1 The Transformer Architecture

Modern language models almost exclusively rely on the **transformer** architecture introduced by Vaswani *et al.* in 2017 [**?**]. The transformer replaces sequential recurrence with self-attention, which lifts a long-standing parallelism bottleneck and lets the model capture dependencies between any two positions in constant computational depth. Figure 2.1 shows the canonical encoder–decoder architecture: the encoder ingests the source sequence and produces a contextualised representation through stacked layers of multi-head self-attention and feed-forward networks; the decoder generates the output sequence autoregressively, combining masked self-attention with cross-attention over the encoder output. Stacking identical blocks yields the encoder, the decoder, or, in the most common contemporary configuration, a *decoder-only* stack specialised for next-token prediction.

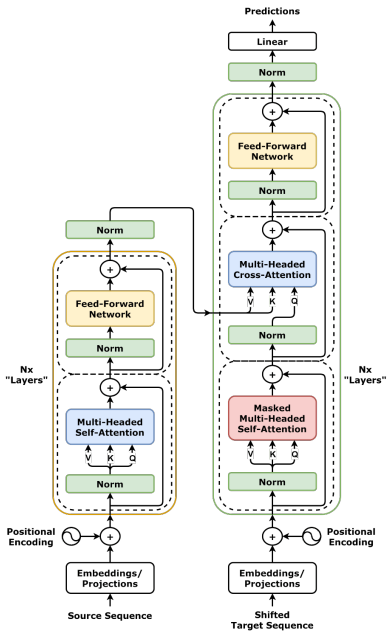

Figure 2.1: Transformer encoder–decoder architecture.

2.3.2 Large Language Models

A *large language model* (LLM) is a transformer-based decoder-only network with billions of parameters, pre-trained self-supervised on a multi-trillion-token corpus to predict the next token, then instruction-tuned to follow human prompts. Open-weight families such as **Llama** and **Qwen** make a class of techniques — inspection, fine-tuning, on-premises deployment — accessible that closed commercial models do not.

2.3.3 Parameter-Efficient Fine-Tuning

Adapting an open-weight LLM to a specialised task no longer requires full fine-tuning of the base model. The *parameter-efficient fine-tuning* (PEFT) family freezes the base weights and trains a small number of additional parameters that capture the task-specific adjustment. The dominant approach is **LoRA** (Low-Rank Adaptation) [**?**], which adds to each linear layer a low-rank update

$$\Delta W = BA, \qquad A \in \mathbb{R}^{r\times d},\ B \in \mathbb{R}^{d\times r},\ r \ll d, \tag{2.2}$$

shrinking the trainable-parameter count by 100 to 1,000× with negligible quality loss. QLoRA extends the recipe by quantising the frozen base weights to four-bit precision, which lets a multi-billion-parameter model be fine-tuned on a single consumer-class GPU.

2.4 Multi-Agent Systems and Agent Protocols

This section defines what an agent is, presents the ReAct loop on which most modern agent implementations are based, surveys the families of multi-agent orchestration frameworks, and closes with the open communication protocols that connect them.

2.4.1 Agent Definition

An *agent* is an autonomous software entity built around a language model that, given a goal expressed in natural language, decides on its own which actions to take, executes them through external *tools* and observes the results before choosing the next step. Three properties distinguish this paradigm from the prompt-and-response one. First, the agent maintains an internal *state* across several steps, which lets it pursue a multi-step plan. Second, it has access to a set of tools — search engines, databases, code interpreters, custom APIs — that ground its reasoning in the external world and allow side effects beyond text generation. Third, it autonomously decides when the task is complete and returns control.

2.4.2 The ReAct Framework

The **ReAct** pattern (*Reason* + *Act*), introduced by Yao *et al.* in 2022 [**?**], is the canonical instantiation of the agent loop. As sketched in Figure 2.2, the model alternates between three kinds of step interleaved in a single autoregressive trace: *Thought*, in which it verbalises its reasoning in natural language; *Action*, in which it emits a structured tool call parsed from its output; and *Observation*, in which the result of the tool call is fed back into the prompt before the next thought. The loop continues until the agent decides to commit to a final answer. Compared with chain-of-thought reasoning alone, the explicit observation channel grounds the chain in factual evidence and substantially reduces hallucination on tasks that require external information.

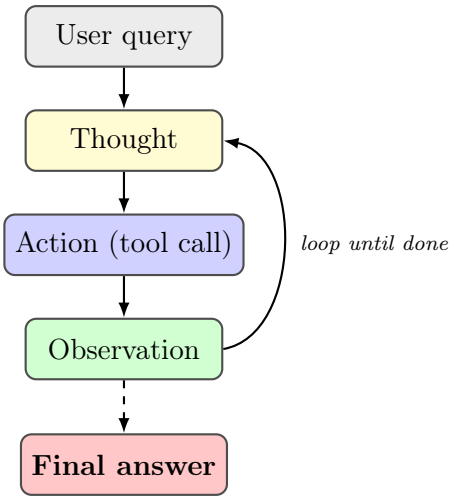

Figure 2.2: ReAct thought–action–observation loop.

2.4.3 Multi-Agent Orchestration

A single agent rarely covers the breadth of a real product. Three families of multi-agent frameworks coexist:

- *Crew-based* frameworks structure the collaboration as a team with explicit roles and a sequential or hierarchical workflow.
- *Conversational* frameworks model agents as participants whose interactions are themselves driven by a language model.
- *State-machine* frameworks express the multi-agent process as a typed graph whose nodes are agent invocations and whose edges are conditional transitions, which makes the dialogue auditable and the branching deterministic.

2.4.4 Agent Communication Contracts

Two open specifications dominate the discussion:

- The *Model Context Protocol* (MCP), introduced in late 2024, describes how a host application exposes tools, resources and prompts to a language-model-driven client.
- The *Agent-to-Agent* (A2A) protocol, proposed in 2025, governs how peer agents discover each other through *AgentCards* and exchange tasks and task results, typically over HTTP and JSON-RPC.

Both protocols decouple agent identity from transport, which makes the message contract an independent design artefact.

2.5 Knowledge Graphs and Graph-Augmented Retrieval

This section defines knowledge graphs, recalls the standard retrieval-augmented-generation recipe and presents Graph RAG, the two grounding mechanisms HireFlow uses to mitigate hallucination.

2.5.1 Knowledge Graphs

A *knowledge graph* (KG) represents domain knowledge as a labelled directed graph in which nodes are entities and edges are typed relationships, traditionally as subject–predicate–object triples inherited from RDF. Knowledge graphs constrain the answers of a generative model to facts present in the graph and expose multi-hop relationships that a flat document index cannot represent.

2.5.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) injects external evidence into the prompt of a language model at inference time. In its simplest form, the corpus is split into chunks, each chunk is embedded with a sentence encoder and indexed in a vector store; at query time, the top-k chunks are retrieved by cosine similarity and concatenated to the prompt. Naive RAG performs well on short factual lookups but degrades on multi-hop questions and on entity-centric queries.

2.5.3 Graph RAG

Graph RAG [**?**] replaces the flat chunk index with a graph of entities and their relationships extracted from the corpus by a language model and clustered into communities. Retrieval becomes a graph traversal that returns a coherent subgraph, possibly augmented with hierarchical community summaries, which measurably outperforms naive RAG on global questions that require integrating information across many documents.

2.6 Candidate–Job Matching

This section traces the progression of candidate–job matching from bag-of-words retrieval to dense semantic similarity, topic-aware hierarchical attention, graph neural networks and constraint-aware shortlisting.

2.6.1 Sentence Embeddings and Semantic Similarity

The first generation of automated candidate screening relied on bag-of-words retrieval (TF-IDF, BM25), which scores a resume against a job description by counting overlapping terms and is therefore blind to synonymy and context. **Sentence-BERT** [?] replaces lexical overlap with semantic similarity through a twin-encoder fine-tuning of BERT that produces fixed-size sentence vectors directly comparable with cosine similarity, and the family has since produced compact and multilingual variants used at scale.

2.6.2 Topic-Aware Person–Job Fit (TAPJFNN)

The *person–job fit* literature frames the matching problem as a binary classification: given a job description and a resume, predict whether the candidate is a good fit. **TAPJFNN** (*Topic-Aware Person–Job Fit Neural Network*) [?] is the state-of-the-art representative of the family. It combines a four-level hierarchical attention with a topic-aware mechanism in which a corpus-level latent topic model modulates the higher attention weights so that the network focuses on the sentences that align with the dominant topic of the job. Figure 2.3 sketches the four-level hierarchy applied symmetrically to the job and to the resume.

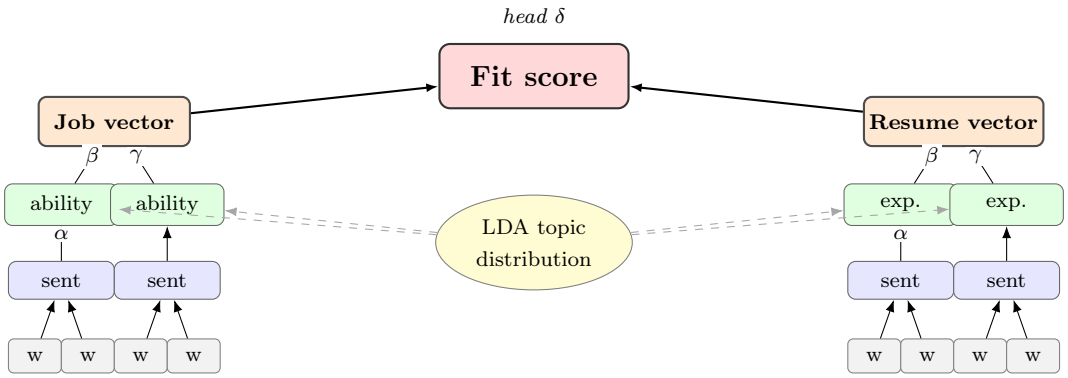

Figure 2.3: TAPJFNN four-level topic-aware hierarchical attention.

2.6.3 Latent Dirichlet Allocation

The topic distributions used by TAPJFNN are produced offline by *Latent Dirichlet Allocation* [?], the canonical generative topic model. LDA assumes that each document is a mixture of a small number of latent topics and that each topic is in turn a distribution over words; the two distributions are inferred jointly through approximate inference.

2.6.4 Graph Neural Networks

A *graph neural network* (GNN) generalises the convolution operator from regular grids to arbitrary graphs. Each node holds a feature vector, and the network repeatedly updates

that vector by aggregating messages from its neighbours, followed by a transformation shared across the graph; after L such layers, the representation of a node summarises an L-hop neighbourhood. Figure 2.4 sketches one round of message passing on a bipartite candidate–job graph. Three operator families dominate: GCN uses a fixed symmetric normalisation, GAT replaces the fixed weights with learned attention coefficients per edge, and GIN uses an injective aggregation that maximises graph-classification power.

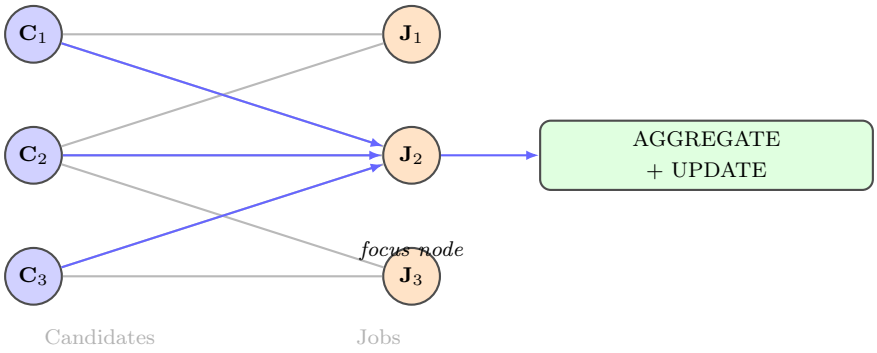

Figure 2.4: GNN message passing on a bipartite candidate–job graph.

2.6.5 Genetic Algorithms for Constrained Shortlisting

A pure ranker returns a list of candidates ordered by predicted fit, but real shortlists must respect a set of operational constraints — a budget on the number of interviews, a diversity target, role quotas and minimum-fit thresholds — that do not factorise across candidates. *Genetic algorithms* are a family of biology-inspired metaheuristics for combinatorial optimisation. A population of candidate solutions is iteratively transformed through three operators:

- *Selection* retains the highest-fitness individuals for the next generation.
- *Crossover* combines two parent solutions into new offspring.
- *Mutation* applies small random changes to maintain diversity.

Constraints are typically enforced through a *death penalty* that assigns zero fitness to infeasible solutions. Algorithm 1 sketches the canonical loop applied to the shortlisting problem.

Algorithm 1 Genetic algorithm for constrained shortlisting

Require: Candidate pool C, fitness f, constraints Φ, population size N, generations T

Ensure: Best feasible shortlist $s^\star$

```
Initialise population P_0 ← {s_1, ..., s_N} at random
for t = 0 to T − 1 do
    for all s ∈ P_t do
        f̂(s) ← f(s) · 𝟙[Φ(s) feasible]                ▷ death penalty
    end for
    P^sel ← SELECT(P_t, f̂)                            ▷ e.g. tournament or roulette
    O ← CROSSOVER(P^sel)
    O ← MUTATE(O)
    P_{t+1} ← top-N individuals of P_t ∪ O by f̂
end for
return arg max_{s∈P_T} f̂(s)
```

2.7 Affective Computing and Speech Technologies

2.7.1 Models of Emotion

The affective-computing community models emotion in two complementary ways:

- A *discrete* set of basic categories (happiness, sadness, fear, anger, surprise, disgust, contempt, neutral), traditionally attributed to Ekman.
- A *continuous* space whose two dominant axes are *valence* (pleasantness) and *arousal* (activation), known as Russell's circumplex.

The two representations coexist in production interview-analysis systems.

2.7.2 Facial Action Units and Landmarks

The FACS decomposes any facial expression into a small set of *Action Units* (AUs) corresponding to individually contractible muscles (AU4 brow lowerer, AU12 lip corner puller). AUs are typically estimated from facial landmarks — several hundred three-dimensional points predicted on the face at thirty frames per second — rather than from raw pixels. Pose-estimation and hand-tracking models complete the perception channel by capturing body language and gesture.

2.7.3 Self-Supervised Speech Encoders

The dominant audio backbone today is **wav2vec 2.0** [?], a self-supervised model that learns speech representations directly from the raw audio waveform. A small downstream

head transforms its contextual embedding into a task-specific output: transcript, emotion class, valence or arousal.

2.7.4 ASR, TTS and Voice-Activity Detection

The interview module relies on three complementary speech components:

- ASR replaces the historical HMM pipeline with end-to-end encoder–decoder transformers trained on weakly supervised audio.
- Neural TTS has moved from autoregressive seq-to-seq systems (Tacotron) to non-autoregressive flow-based models such as VITS that run faster than real time on commodity CPUs.
- VAD decides whether speech is present in a short audio window; modern neural VAD replaces the WebRTC filter with a small recurrent network.

2.8 Personalised Ranking

A single global ranker is a poor approximation of any one recruiter's preferences, since two recruiters weight cultural fit, technical depth and communication style differently. The most pragmatic personalisation model for a high-stakes hiring use case is *logistic regression*: its coefficients are directly readable as feature weights and its calibrated probability output admits a natural confidence interpretation, which matters when the system operates under regulatory constraints.

2.9 Conclusion

This chapter laid the conceptual foundations of the work and surveyed the families of techniques on which the rest of the report builds. The next chapters detail, sprint by sprint, the design and implementation of the system that integrates them.

Chapter 3

Analysis and Design

Introduction

The previous chapters introduced the host organisation, motivated the project and surveyed the state of the art on which HireFlow is built. The present chapter turns this groundwork into a concrete engineering plan. It first captures the functional and non-functional requirements through actor identification, a global use-case diagram and a quality matrix. It then frames the work in a Scrum cadence with a product backlog and a six-sprint roadmap. Finally, it lays out the physical, logical and multi-agent architectures of the platform together with the hardware and software environment that supports its delivery.

3.1 Requirements Analysis

This section enumerates who the platform serves, what it must do and under which quality constraints it must operate.

3.1.1 Identification of the Different Actors

Two categories of human actor interact with HireFlow, complemented by a handful of external technical actors whose behaviour the system has to integrate with.

- **Recruiter** (primary): opens vacancies, drafts and validates job descriptions, triggers candidate matching, launches interviews, reviews scoring reports and refines the personalisation signal. Most of the platform's value is delivered to this actor.
- **Candidate** (primary): applies to a vacancy, uploads a CV, takes the conversational interview and consults the outcome of the application. Interactions are designed to be transparent and explainable.

- **External AI services** (supporting): the vLLM server, the Groq inference endpoint, and the speech models (Kokoro TTS, Moonshine ASR) participate as supporting actors invoked synchronously by the backend during the interview and scoring flows.

3.1.2 Functional Requirements

Functional requirements are organised by epic; each line is uniquely identified so that traceability with the use-case model, the product backlog and the test plan can be enforced.

ID	Epic	Requirement
FR-01	JD authoring	Generate a culture-aware JD from a short brief.
FR-02	JD authoring	Retrieve a relevant subgraph from the talent knowledge graph (Graph RAG).
FR-03	JD authoring	Allow the recruiter to edit and validate the draft.
FR-04	Matching	Parse uploaded CVs into a structured skill profile.
FR-05	Matching	Rank candidates against a JD using a deep GNN matcher.
FR-06	Matching	Display per-candidate evidence supporting each rank.
FR-07	Interview	Conduct a voice-driven adaptive interview grounded in the CV.
FR-08	Interview	Render a Three.js avatar with lip-synced speech.
FR-09	Interview	Choose between follow-up and topic-switch based on answer depth.
FR-10	Behavioural analysis	Capture face, pose and audio features in real time.
FR-11	Behavioural analysis	Fuse multimodal features into the four behavioural signals.
FR-12	Scoring	Produce a per-dimension scoring report with an explanation paragraph.
FR-13	Scoring	Export the report as a downloadable PDF.
FR-14	Personalisation	Learn a per-recruiter taste model from historical decisions.
FR-15	Personalisation	Surface the taste signal alongside the central recommendation.

Table 3.1: Functional requirements traced by epic.

3.1.3 General Use Case Diagram

The diagram in Figure 3.1 summarises the actors and the high-level capabilities they exercise, together with the behavioural-signal capture that is factored out as an included use case.

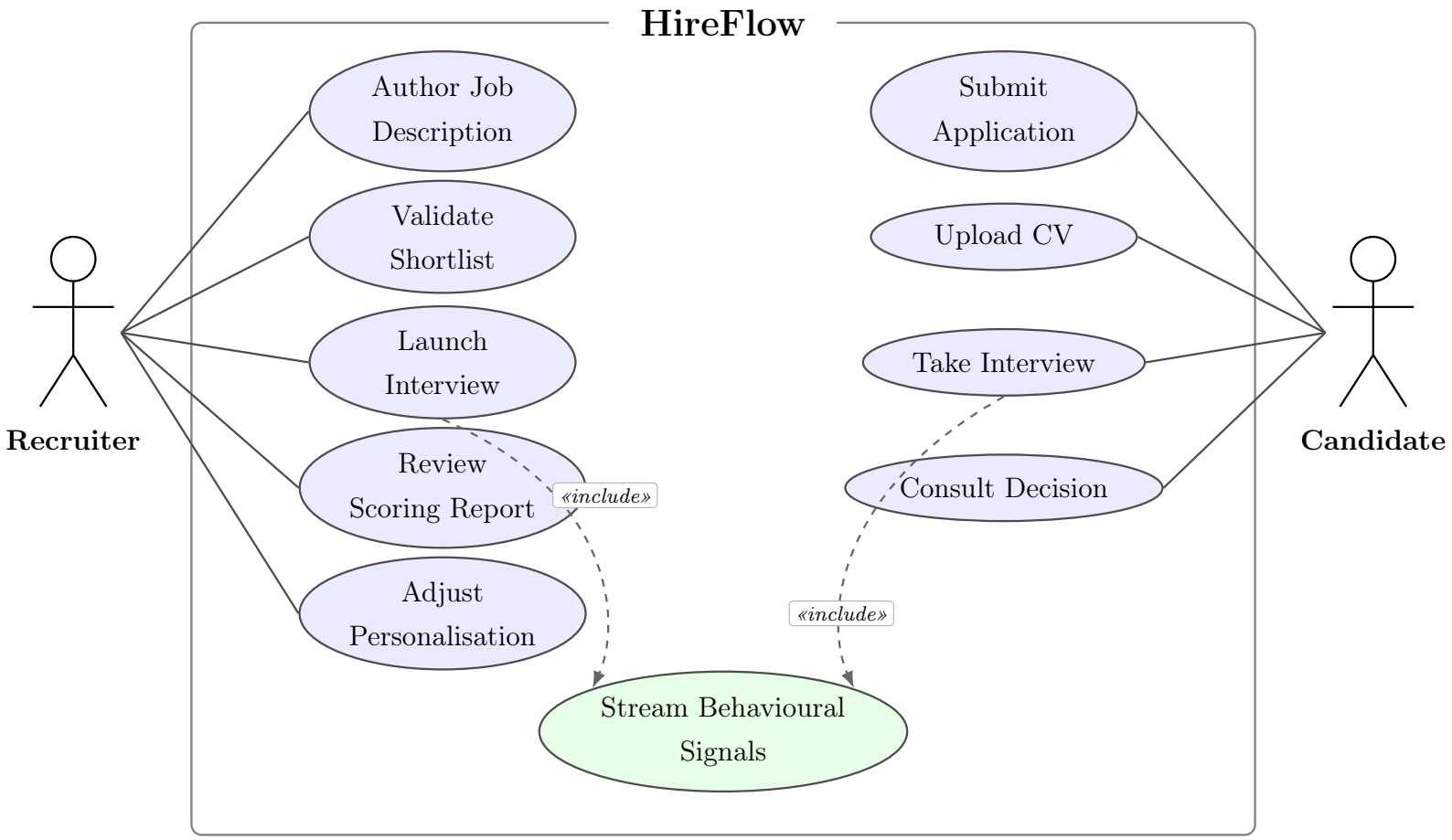

Figure 3.1: General use-case diagram of HireFlow.

The diagram makes the responsibility split visible. The recruiter drives every authoring, evaluation and personalisation activity, while the candidate is only exposed to the surfaces strictly needed to apply, interview and consult the outcome. The *«include»* relationships factor out the behavioural-signal capture that both *Launch Interview* and *Take Interview* trigger.

3.1.4 Non-Functional Requirements

The non-functional requirements derive from the dual nature of the platform: a research-grade multi-agent system that must nevertheless behave as a usable product.

- **Performance**: end-to-end conversational latency below two seconds per turn; behavioural-signal pipeline running at no less than fifteen frames per second on a single GPU.
- **Scalability**: stateless agent contracts that allow additional model instances to be added behind a load balancer without code changes.
- **Reliability**: graceful degradation if a single agent fails — the recruiter must always reach a usable shortlist even when the conversational layer is unavailable.
- **Observability**: every agent emits structured logs capturing the inputs, the version of the model that produced the decision and the timestamp at which it was emitted.

- **Usability**: a recruiter with no AI background must be able to author a JD, launch an interview and read the scoring report without supervision.
- **Maintainability**: each agent is a self-contained Python package with its own tests and contract so that components can evolve independently.
- **Portability**: the platform runs on a developer laptop and on a single GPU node, without any external orchestrator dependency.

3.2 Project Planning with Scrum

HireFlow was delivered following a Scrum cadence adapted to the academic context of an end-of-studies project. The methodology was chosen because it accommodates the exploratory nature of multi-agent AI work — the precise shape of each agent emerges as data and models are explored — while still imposing the discipline of two- to three-week sprints, a written backlog and a demonstrable increment at the end of every iteration.

3.2.1 Definition of Roles

The project assigned the three canonical Scrum roles to the actors of the internship, with each role mapped to a concrete responsibility inside the team.

Role	**Responsibility and assignment**
Product Owner	Owns the product vision, prioritises the backlog and validates each sprint demo. Assigned to the Talan Tunisie Consulting professional supervisor.
Scrum Master	Facilitates the ceremonies, removes impediments and protects the team's focus. Assigned jointly to the academic supervisor at ENSI and the professional mentor at Talan.
Development Team	Designs, implements and validates each increment. Composed of the PFE student acting as a single full-stack AI developer.

Table 3.2: Scrum roles and their assignment.

Three ceremonies were observed throughout the internship: a weekly sprint review with the Product Owner, a fortnightly sprint planning session, and a daily written stand-up logged in the project tracker. The Definition of Done required, for every user story, working code merged on the main branch, unit tests passing in continuous integration, a short demo recording and a paragraph appended to the engineering log.

3.2.2 Product Backlog

The product backlog organises the functional requirements of Section 3.1.2 into user stories. Each story follows the canonical *As a / I want / so that* template and carries a MoSCoW priority. Story points are estimated on a Fibonacci scale and indicate complexity rather than calendar effort.

ID	Epic	User story	Pts	Prio.
US-01	JD authoring	As a recruiter, I want to generate a culture-aware JD from a short brief so that I save authoring time.	8	M
US-02	JD authoring	As a recruiter, I want the agent to consult a talent knowledge graph (Graph RAG) so that the JD reflects the company's skill landscape.	5	M
US-03	JD authoring	As a recruiter, I want to edit and validate the draft so that I keep control on the final wording.	2	M
US-04	Matching	As a recruiter, I want uploaded CVs parsed into a structured skill profile so that matching becomes possible.	5	M
US-05	Matching	As a recruiter, I want candidates ranked against a JD using a deep GNN matcher so that I focus on the most relevant profiles.	8	M
US-06	Matching	As a recruiter, I want the evidence behind each rank so that I can trust and defend the recommendation.	3	M
US-07	Interview	As a candidate, I want a voice-driven adaptive interview so that the conversation feels natural.	8	M
US-08	Interview	As a candidate, I want to see a Three.js avatar that lip-syncs the questions so that the exchange is engaging.	5	S
US-09	Interview	As a recruiter, I want the agent to decide between follow-up and topic-switch so that depth is preserved.	5	M
US-10	Behavioural	As a recruiter, I want face, pose and audio features captured in real time so that behavioural signals can be computed.	5	M
US-11	Behavioural	As a recruiter, I want the four behavioural signals fused into a profile so that the scoring report is enriched.	5	M
US-12	Scoring	As a recruiter, I want a per-dimension scoring report with explanations so that I understand each verdict.	5	M
US-13	Scoring	As a recruiter, I want to export the report as a PDF so that I can share it with the hiring panel.	2	S
US-14	Personalisation	As a recruiter, I want a private taste model learned from my decisions so that suggestions adapt to my style.	8	C
US-15	Personalisation	As a recruiter, I want the taste signal displayed next to the central rank so that I keep oversight.	3	C

Table 3.3: Product backlog (M = Must, S = Should, C = Could).

3.2.3 Sprint Planning

The fifteen stories were grouped into six sprints by thematic proximity rather than strict priority order, so that each iteration delivered a self-contained increment that the Product Owner could demonstrate end-to-end. Two of the sprints — Sprint 1 and Sprint 4 — carry an explicit CRISP-DM cycle because they involve a modelling experiment whose result feeds the rest of the platform.

#	**Theme**	**Dur.**	**Stories**	**Methodology**
S1	LLM fine-tuning for the recruiter persona	3w	US-07, US-09	Scrum + CRISP-DM
S2	Conversational interview	3w	US-07, US-08, US-09	Scrum
S3	Computer-vision component and avatar	3w	US-08, US-10, US-11	Scrum
S4	Job matching	3w	US-04, US-05, US-06	Scrum + CRISP-DM
S5	Culture-aware job description	2w	US-01, US-02, US-03	Scrum
S6	Taste module	2w	US-14, US-15	Scrum

Table 3.4: Six-sprint roadmap of the HireFlow internship.

The roadmap is deliberately bottom-up. The persona-fine-tuned LLM produced in Sprint 1 underpins both the conversational interview of Sprint 2 and the culture-aware JD authoring of Sprint 5; the matching back-end built in Sprint 4 is reused by the personalisation layer of Sprint 6. This ordering keeps the demos incremental and lets each sprint be evaluated against a working baseline rather than against a moving target.

3.3 Application Architecture

The platform is built as a single deployable unit that nevertheless respects three classical architectural views. Section 3.3.1 fixes the **physical** topology — which artefacts run on which machine, with which protocols. Section 3.3.2 then reads the same system through the **logical** lens of a three-tier web application: presentation, business, data. Section 3.3.3 zooms into the business tier to expose the **multi-agent** composition that gives HireFlow its specificity.

3.3.1 Physical Architecture

The deployment is intentionally compact: one developer client, one GPU-equipped server, and one external inference provider for fall-back generation. This keeps the platform

reproducible on a single Lightning AI Studio node, which was the constraint imposed by the academic environment.

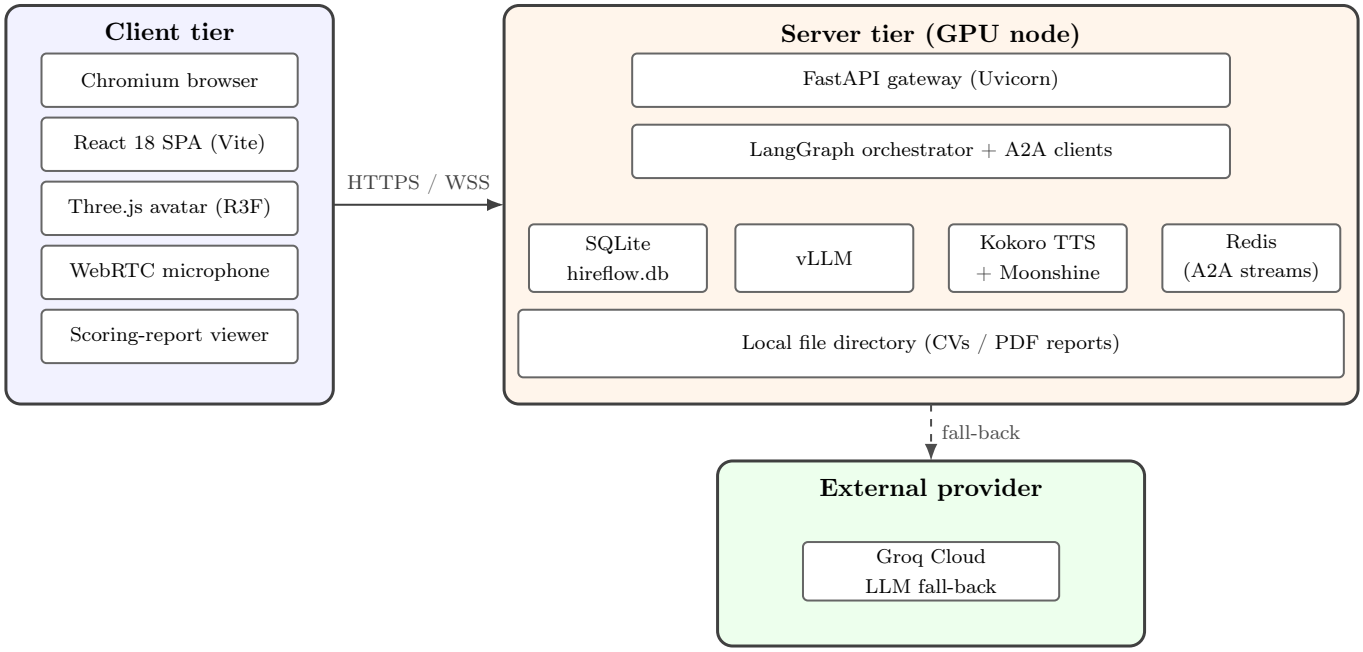

Figure 3.2: Physical deployment architecture of HireFlow.

The client requires nothing more than a recent browser supporting WebGL and WebRTC. All trained weights, the SQLite store and the object directory live on the server, which means that no candidate data ever leaves the perimeter under normal operation. The dashed arrow towards Groq represents a fall-back path used only when the local vLLM engine is unavailable; that path is gated behind a feature flag so it can be disabled when stricter data-residency constraints apply.

3.3.2 System Logical Architecture

Mapped onto the classical three-tier model, the platform's responsibilities partition cleanly into a presentation tier, a business tier and a data tier. The business tier is itself the locus of the multi-agent system that Section 3.3.3 details.

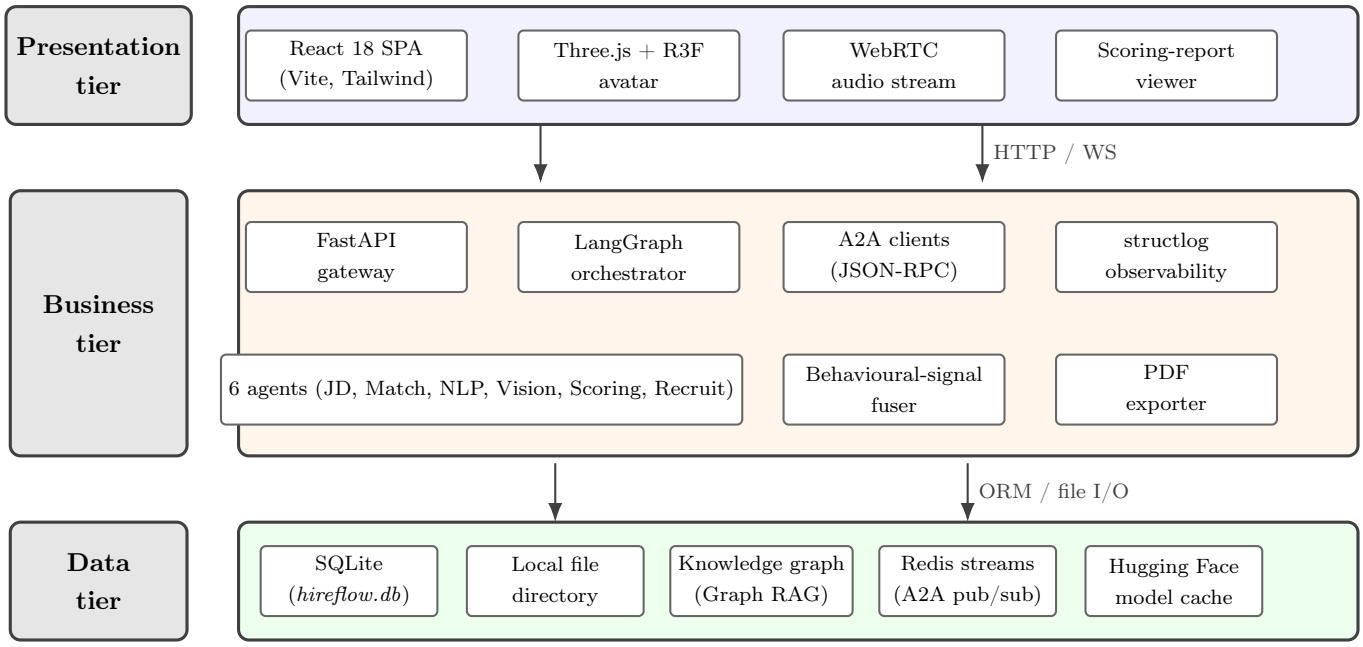

Figure 3.3: Three-tier logical architecture of HireFlow.

The presentation tier is a single-page application built with React and styled with Tailwind. It hosts a Three.js avatar through React Three Fiber and streams the candidate's microphone through WebRTC to the backend. The business tier exposes a FastAPI gateway that dispatches each call to the relevant agent through a LangGraph state machine, with inter-agent traffic carried over Redis streams that follow the A2A contract. The data tier holds a SQLite file for transactional state, a local file directory for binary assets, an in-process talent knowledge graph queried by the JD agent through Graph RAG, and a shared Hugging Face cache for model weights.

3.3.3 Logical Architecture of the Multi-Agent System

Inside the business tier, HireFlow follows a hub-and-spoke multi-agent pattern. The **Recruit Agent** acts as the orchestrator: it receives the recruiter's intent, decomposes it into specialised subtasks and routes each subtask to one of the five specialist agents. Communication between agents follows the **A2A** protocol over JSON-RPC and is materialised on top of Redis streams, while each agent embeds its own models and tools (the talent knowledge graph queried through Graph RAG, vLLM, the GNN matcher, MediaPipe, Kokoro TTS, Moonshine) directly.

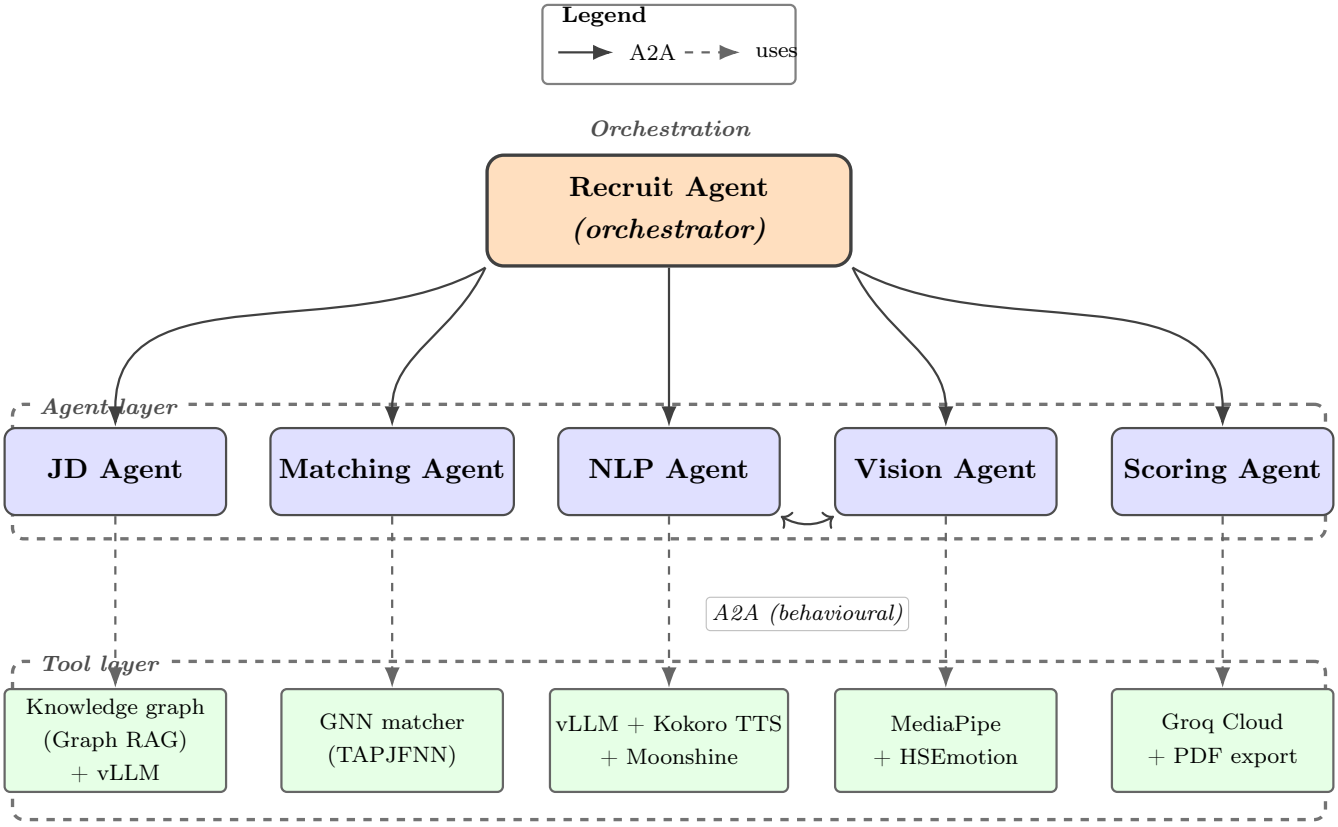

Figure 3.4: Logical architecture of the HireFlow multi-agent system.

The shape of the diagram captures three design intentions. First, **specialisation**: every agent owns a single concern — JD authoring, semantic matching, conversational NLP, behavioural vision, or scoring — which keeps prompts compact and weights swappable. Second, **co-located tooling**: each agent embeds its own models and runs in-process, so the deployment stays a single GPU node with no external broker beyond Redis. Third, **lateral communication**: the NLP and Vision agents talk directly to each other during an interview turn, because the orchestrator does not need to mediate the per-frame stream that fuses speech with behavioural cues.

3.4 Work Environment

This section lists the hardware and software stack that supported the implementation, the training experiments and the writing of the present report.

3.4.1 Hardware Environment

Two machines were used throughout the internship: a developer laptop for everyday coding and report writing, and a Lightning AI Studio GPU node for training experiments and inference benchmarking. Table 3.5 contrasts their specifications.

Component	**Developer laptop**	**Lightning AI Studio**
CPU	AMD Ryzen 5 7535HS, 6 cores	16 vCPU
GPU	NVIDIA RTX 2050, 4 GB VRAM	NVIDIA RTX 6000 Pro, 96 GB VRAM
RAM	16 GB	32 GB
Storage	512 GB SSD	200 GB persistent volume
Connectivity	100 Mbps	1 Gbps
Role	Coding, debugging, report	Training, inference, demos

Table 3.5: Hardware environment used during the internship.

3.4.2 Software Environment

The software stack is open-source end to end. The choices favour mature, well-documented projects with active communities so that the platform remains maintainable beyond the duration of the internship. Table 3.6 groups the tools by responsibility.

Layer	**Tools and libraries**
Operating system	Ubuntu 22.04 LTS (server), Zorin OS (laptop)
Languages	Python 3.11, TypeScript 5, SQL
Backend	FastAPI, Uvicorn, Pydantic v2, sqlite3, structlog
AI/ML	PyTorch 2, Hugging Face Transformers, PEFT/LoRA, vLLM, LangGraph
Voice	Kokoro TTS, Moonshine, Silero VAD
Vision	MediaPipe, OpenCV, HSEmotion, Wav2Vec2
Frontend	React 19, Vite 7, TailwindCSS 4, Three.js, R3F, Drei
Persistence	SQLite 3, Redis streams, local file directory
External APIs	Groq Cloud (scoring LLM), Hugging Face Hub (weights)
IDE & tooling	Visual Studio Code, Git, GitHub
Documentation	LaTeX (TeXLive 2024), draw.io, TikZ

Table 3.6: Software environment used during the internship.

Conclusion

This chapter translated HireFlow's high-level objectives into a concrete engineering plan. The functional and non-functional requirements were enumerated and exposed through a global use-case diagram; a Scrum cadence with a fifteen-story product backlog and a six-sprint roadmap was framed; and the physical, three-tier logical and multi-agent architectures were detailed alongside the hardware and software environment that supports them. The next chapter reports on the realisation of these models and on the experimental validation of each agent.

Conclusion et perspectives

Récapitulation du travail réalisé

Enrichissements possibles.

Chapter A

Annexe 1

Netography

[URL1] Cross-industry standard process for data mining (crisp-dm). Wikimedia Commons, https://commons.wikimedia.org/wiki/File:CRISP-DM_Process_Diagram.png, consulted on 2026-05-07.

[URL2] Eightfold ai – talent intelligence platform. https://eightfold.ai/, consulted on 2026-04-15.

[URL3] Hirevue – hiring experience platform. https://www.hirevue.com/, consulted on 2026-04-15.

[URL4] Linkedin recruiter – find and engage talent. https://business.linkedin.com/talent-solutions/recruiter, consulted on 2026-04-15.

[URL5] Manatal – ai recruitment software. https://www.manatal.com/, consulted on 2026-04-15.

[URL6] Scrum in software engineering – miquido. https://www.miquido.com/blog/scrum-in-software-engineering/, consulted on 2026-05-07.

[URL7] Talan – positive innovation. https://www.talan.com/, consulted on 2026-04-15.